

Resumen: Modelo de Análisis en Diseño de Sistemas (UTN)

El flujo de trabajo (Workflow) de Análisis toma como entrada el **Modelo de Requerimientos** (Casos de Uso, Modelo de Dominio, Glosario) y produce como salida el **Modelo de Análisis**. Este modelo es el "cómo" lógico y se divide en dos grandes vistas:

- **Vista Estática:** Representada por el Diagrama de Clases de Análisis. Nos muestra la estructura.
- **Vista Dinámica:** Representada por los Diagramas de Secuencia, Comunicación y Máquina de Estados. Nos muestra cómo los objetos colaboran a lo largo del tiempo.

1. Las Clases del Análisis (El modelo ECB)

Para separar responsabilidades, en la cátedra utilizamos tres tipos de clases (estereotipos):

Estereotipo	Descripción	Ejemplo
Entidad (Entity)	Modela la información que es persistente (sobrevive a la ejecución del sistema).	Festival, Pedido, Factura
Límite (Boundary)	Modela la interfaz entre el sistema y el ambiente (el actor). Maneja entradas y salidas.	PantallaAdministrarFestival
Control (Control)	Clase de <i>Fabricación Pura</i> . Coordina y delega la ejecución de un caso de uso.	GestorFestival

2. Asignación de Responsabilidades: Patrones GRASP

GRASP (General Responsibility Assignment Software Patterns) nos da reglas de oro para decidir qué objeto hace qué.

- **Experto en Información:** "Lo hace quien lo conoce". Asigna la responsabilidad a la clase que tiene los datos necesarios para cumplirla.
- **Creador:** Asigna a la clase B la responsabilidad de crear una instancia de A si: B contiene a A, B agrega a A, o B tiene los datos de inicialización de A.
- **Controlador:** Maneja los eventos del sistema. Evita que la capa de presentación tenga lógica de negocio.
- **Bajo Acoplamiento & Alta Cohesión:** Principios evaluativos. Clases que hacen una sola cosa bien y dependen de la menor cantidad de clases posibles.

3. Análisis Profundo de Diagramas UML

A. Diagrama de Secuencia (Vista Dinámica)

Muestra la interacción de un conjunto de objetos a lo largo del tiempo para realizar un Caso de Uso.

- **Actor:** Inicia la interacción enviando un mensaje síncrono a la Boundary.
- **Línea de vida:** La línea punteada vertical que representa la existencia del objeto.
- **Foco de control (Cilindro):** Indica cuándo el objeto está ejecutando una acción.
- **Mensaje Síncrono (Flecha llena):** El emisor espera la respuesta antes de seguir.
- **Fragmentos (Alt, Loop, Opt):** Controlan el flujo. Loop (iterar), Alt (If-Else), Opt (If simple).
- **Creación (new()):** Apunta directamente a la "caja" del objeto nuevo.

B. Diagrama de Máquina de Estados (Vista Dinámica)

Especifica la secuencia de estados por los que pasa un objeto durante su vida en respuesta a eventos. **SOLO** se usa en clases de Entidad cuyo comportamiento varía radicalmente según su estado.

- **Estado:** Atributo + Valor en un momento del tiempo.
- **Transición:** El paso de un estado a otro.
- **Evento Disparador:** El método o caso de uso que provoca el cambio de estado.
- **Condición de Control (Guarda):** Condición booleana [Condición].

4. Explicaciones Ampliadas: Conceptos Difíciles

Visibilidad (Por Atributo vs. Por Parámetro)

Tipo de Visibilidad	Significado	Representación UML
Por Atributo	El objeto origen guarda una referencia permanente al destino.	Asociación o Agregación
Por Parámetro	El origen solo conoce al destino durante la ejecución de un método.	Dependencia (flecha punteada)

Modificación Masiva de Objetos (El "Loop" en Secuencia)

Paso a paso lógico del Gestor al modificar muchos objetos:

1. Solicita el dato a la pantalla.
2. Inicia un fragmento Loop para recorrer la colección de objetos.
3. Envía un método de consulta (ej. esTuMarca()) (Patrón Experto).
4. Si responde True (fragmento Opt o Alt), el Gestor llama al método de actualización de la entidad.

5. Reglas de Oro y Relaciones Clave

- **Regla de Capas Estricta:** Actor → Boundary → Control → Entity. Ninguna se puede saltar ni ir en desorden.
- **Regla del Experto:** El Gestor no calcula por su cuenta, delega el cálculo a la Entidad que tiene los datos.
- **Regla de Creación en Cascada:** El "Todo" crea a sus "Partes" (Creador).

6. Errores Típicos que Cuestan Puntos

- **Resolver reglas de negocio en la capa de presentación:** Poner a la pantalla (Boundary) a sumar precios o validar lógicas.
- **El "Gestor Dios":** Hacer que el Controlador (Gestor) haga todos los cálculos dejando a las Entidades como meras estructuras de datos.
- **Flecha de creación mal dibujada:** Hacer que un mensaje de creación (new()) impacte en la línea de vida en vez de en la caja principal del objeto.
- **Máquinas de estado innecesarias:** Hacerle estados a clases que son simples diccionarios o descriptores de datos sin comportamiento dinámico.

7. Posibles Preguntas de Parcial / Final

¿Por qué el Controlador delega la creación de un detalle a la cabecera?

Para mantener el bajo acoplamiento. Según el patrón Creador de GRASP, la cabecera contiene a los detalles, por lo que es la "Experta" en crearlos. Si el Gestor lo hiciera, se acoplaría a las partes internas de la cabecera.

¿Qué indica que una clase requiere una Máquina de Estados?

Cuando los objetos tienen un comportamiento que depende de su estado. Es decir, cuando la respuesta a un mismo evento varía según los valores actuales de sus atributos.

8. Mini Resumen Final (Cheat Sheet)

* Workflow: Requerimientos → Análisis → Diseño → Implementación.

* Clases ECB: Límite (Pantalla) → Control (Gestor) → Entidad (Dominio).

* GRASP:

- Experto: Delega al que tiene los atributos.

- Creador: El "Todo" crea a la "Parte".

- Controlador: Gestor que protege al dominio de la pantalla.

* Secuencia: Lee temporalmente. Flecha llena = sincrónico. Línea de vida = objeto vivo.

* Consistencia: ¡Método mandado en Secuencia = Método anotado en el Diagrama de Clases!

